

Linux Namespace

- Network Namespace is a separate network inside the host.
 - Host and Namespace doesn't know anything about each other.
- The namespace will have its own *network interfaces*, *arp table*, *routing table* (independent of Host).
- I've 2 vms: vm1, vm2
 - vm1: 10.10.10.10/24**
 - vm2: 10.10.10.11/24**
- vm1 and vm2 can ping each other because both belong to same LAN (10.10.10.x).

```
vagrant@vm1:~$ route
Kernel IP routing table
Destination Gateway Genmask Flags Metric Ref Use Iface
default 10.0.2.2 0.0.0.0 UG 100 0 0 enp0s3
10.0.2.0 0.0.0.0 255.255.255.0 U 100 0 0 enp0s3
10.0.2.2 0.0.0.0 255.255.255.255 UH 100 0 0 enp0s3
10.10.10.0 0.0.0.0 255.255.255.0 U 0 0 0 enp0s8
192.168.0.1 10.0.2.2 255.255.255.255 UGH 100 0 0 enp0s3
```

route table]

Gateway 0.0.0.0 means this subnet (10.10.10.0) Gateway/Router is not needed because both belong to same **subnet**.

10.10.10.0 is added on **enpos8** (means this interface is connected to private network i.e. vm to vm communication)

- ip add**

display all the network interfaces names like *lo* (loopback), *enpos3*, *enpos8* ..etc.

en: ethernet; **po**: PCI bus 0; **s3**: slot 3

enpos8: ethernet device on slot 8

in out VM (vm1, vm2), *lo enpos3 enpos8* will be there.

s3 is for vm-to-vm communication (inter LAN), **s8** is for internet access (NAT).

- To create a namespace

ip netns add <namespace name>

- We'll create 2 namespaces having name **red**, **blue** .

```
root@vm1:~# ip netns add red
root@vm1:~# ip netns add blue
```

↗ **ip netns list**

to list all the namespaces

```
root@vm1:~# ip netns list
blue
red
```

々 To execute any command inside a namespace

ip netns exec <network namespace name> <command to execute>

↗ **ip netns exec red ip add**

```
root@vm1:~# ip netns exec red ip add
1: lo: <LOOPBACK> mtu 65536 qdisc noop state DOWN group default
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
```

[only loopback is present]

[same for **blue** namespace as well]

↗ **ip netns exec red route**

```
root@vm1:~# ip netns exec red ip route
root@vm1:~#
```

[no route is present]

[same for **blue** namespace as well]

々 Now we'll create a **veth pair** (just like a virtual cable) to connect the 2 namespaces.

ip link add <veth's one end name> **type veth peer name** <veth's another end name>

[when you create a **veth pair**, it creates 2 **veth** network interface in the host and connect them internally. From outside, it looks 2 ends of a virtual ethernet cable]

↗ **ip link add veth-red type veth peer name veth-blue**

```
valid_etc forever preferred_etc forever
4: veth-blue@veth-red: <BROADCAST,MULTICAST,M-DOWN> mtu 1500 qdisc noop state DOWN group default qlen 1000
    link/ether ba:86:30:88:3b:b8 brd ff:ff:ff:ff:ff:ff
5: veth-red@veth-blue: <BROADCAST,MULTICAST,M-DOWN> mtu 1500 qdisc noop state DOWN group default qlen 1000
    link/ether de:62:4d:8e:da:b9 brd ff:ff:ff:ff:ff:ff
```

[**ip add**]

[2 veth network interface; internally connected; both represents one end]

veth-blue@veth-red -> veth-blue interface is connected to veth-red

veth-red@veth-blue -> veth-red interface is connected to veth-blue

↗ If both the ends of a veth pair present in same place (ex: both in host), then it is displayed like

`end1-name@end2-name` (end1 to end2)

`end2-name@end1-name` (end2 to end1)

if both are present in different namespaces (host & namespace; or namespace1 & namespace2), then it'll be displayed like

`end1-name@if<end2's index at its namespace>`

`end2-name@if<end1's index at its namespace>`

々 To connect **veth** ends to namespaces

ip link set <veth end name> **netns** <network namespace name>

々 Connect *veth-red* end to the namespace **red**

ip link set *veth-red* **netns** *red*

⌘ Now if you see,

```
4: veth-blue@if5: <BROADCAST,MULTICAST> mtu 1500 qdisc noop state DOWN group default qlen 1000
    link/ether ba:86:30:88:3b:b8 brd ff:ff:ff:ff:ff:ff link-netns red
```

link-netns red it means other end is present in namespace *red*.

⌘ *veth-red@veth-blue* is gone

because *veth-red* is now connected to a namespace. Host linux (host) cannot show peer interface directly it moved to namespace.

⌘ *veth-blue@veth-red* got updated to *veth-blue@if5* because *veth-red*'s index inside namespace **red** is **5**.

```
root@vm1:~# ip netns exec red ip add
1: lo: <LOOPBACK> mtu 65536 qdisc noop state DOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
5: veth-red@if4: <BROADCAST,MULTICAST> mtu 1500 qdisc noop state DOWN group default qlen 1000
    link/ether de:62:4d:8e:da:b9 brd ff:ff:ff:ff:ff:ff link-netnsid 0
```

[**ip netns exec red ip add**]

And also, here, *veth-red@if4* because *veth-blue* is at 4th index .

link-netnsid 0 means other end present in host.

々 Connect *veth-blue* to the namespace **blue**

ip link set *veth-blue* **netns** *blue*

⌘ Now, *veth-blue* will be gone from host because it belongs to namespace *blue*.

々 Network interfaces in both namespace now

⌘ Namespace **red**:

ip netns exec red ip add

```

root@vm1:~# ip netns exec red ip add
1: lo: <LOOPBACK> mtu 65536 qdisc noop state DOWN group default qlen 1000
   link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
5: veth-red@if4: <BROADCAST,MULTICAST> mtu 1500 qdisc noop state DOWN group default qlen 1000
   link/ether de:62:4d:8e:da:b9 brd ff:ff:ff:ff:ff:ff link-netns blue

```

• **link-netns blue** : other end is in namespace blue

• Namespace **blue**:

ip netns exec blue ip add

```

root@vm1:~# ip netns exec blue ip add
1: lo: <LOOPBACK> mtu 65536 qdisc noop state DOWN group default qlen 1000
   link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
4: veth-blue@if5: <BROADCAST,MULTICAST> mtu 1500 qdisc noop state DOWN group default qlen 1000
   link/ether ba:86:30:88:3b:b8 brd ff:ff:ff:ff:ff:ff link-netns red

```

• **link-netns red** : other end is in namespace red

✧ Now make the network interfaces in both namespaces UP.

ip netns exec <network namespace name> ip link set <interface name> up

• **ip netns exec red ip link set lo up**

ip netns exec red ip link set veth-red up

```

root@vm1:~# ip netns exec red ip add
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
   link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
   inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
   inet6 ::1/128 scope host
       valid_lft forever preferred_lft forever
5: veth-red@if4: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc noqueue state LOWERLAYERDOWN group default qlen 1000
   link/ether de:62:4d:8e:da:b9 brd ff:ff:ff:ff:ff:ff link-netns blue

```

LOWERLAYERDOWN: means it is up, but its other end (veth-blue) is down.

• **ip netns exec red ip link set lo up**

ip netns exec red ip link set veth-red up

```

root@vm1:~# ip netns exec blue ip add
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
   link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
   inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
   inet6 ::1/128 scope host
       valid_lft forever preferred_lft forever
4: veth-blue@if5: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default qlen 1000
   link/ether ba:86:30:88:3b:b8 brd ff:ff:ff:ff:ff:ff link-netns red
   inet6 fe80::b886:30ff:fe88:3bb8/64 scope link
       valid_lft forever preferred_lft forever

```

• Now, both *veth-red* and *veth-blue* are UP.

✧ Assign IP to both the namespace

ip netns exec <namespace name> ip addr add <IP>/<subnet mask length>

dev <interface name>

- Assign in namespace **red**:

`ip netns exec red ip addr add 192.168.15.2/24 dev veth-red`

```
root@vm1:~# ip netns exec red ip add
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN gr
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
        valid_lft forever preferred_lft forever
5: veth-red@if4: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqu
    link/ether de:62:4d:8e:da:b9 brd ff:ff:ff:ff:ff:ff link-netns blue
    inet 192.168.15.2/24 scope global veth-red
        valid_lft forever preferred_lft forever
    inet6 fe80::dc62:4dff:fe8e:dab9/64 scope link
        valid_lft forever preferred_lft forever
```

[IP is assigned]

- Assign in namespace **blue**:

`ip netns exec blue ip addr add 192.168.15.3/24 dev veth-blue`

```
root@vm1:~# ip netns exec blue ip add
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN g
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
        valid_lft forever preferred_lft forever
4: veth-blue@if5: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc no
    link/ether ba:86:30:88:3b:b8 brd ff:ff:ff:ff:ff:ff link-netns red
    inet 192.168.15.3/24 scope global veth-blue
        valid_lft forever preferred_lft forever
    inet6 fe80::b886:30ff:fe88:3bb8/64 scope link
        valid_lft forever preferred_lft forever
```

- The IPs are 192.168.15.2/24 & 192.168.15.3/24,
so private LAN's network IP will be: **198.168.15.0**
& subnet mask will be: **255.255.255.0**

🔗 Ping the namespaces from each other. It'll work fine.

- From namespace **red**:

`ip netns exec red ping -c 4 192.168.15.3`

```
root@vm1:~# ip netns exec red ping -c 4 192.168.15.3
PING 192.168.15.3 (192.168.15.3) 56(84) bytes of data.
 64 bytes from 192.168.15.3: icmp_seq=1 ttl=64 time=0.041 ms
 64 bytes from 192.168.15.3: icmp_seq=2 ttl=64 time=0.135 ms
 64 bytes from 192.168.15.3: icmp_seq=3 ttl=64 time=0.060 ms
 64 bytes from 192.168.15.3: icmp_seq=4 ttl=64 time=0.060 ms

--- 192.168.15.3 ping statistics ---
 4 packets transmitted, 4 received, 0% packet loss, time 3071ms
 rtt min/avg/max/mdev = 0.041/0.074/0.135/0.036 ms
```

- From namespace **blue**:

`ip netns exec blue ping -c 4 192.168.15.2`

```
root@vm1:~# ip netns exec blue ping -c 4 192.168.15.2
PING 192.168.15.2 (192.168.15.2) 56(84) bytes of data.
64 bytes from 192.168.15.2: icmp_seq=1 ttl=64 time=0.210 ms
64 bytes from 192.168.15.2: icmp_seq=2 ttl=64 time=0.091 ms
64 bytes from 192.168.15.2: icmp_seq=3 ttl=64 time=0.091 ms
64 bytes from 192.168.15.2: icmp_seq=4 ttl=64 time=0.057 ms

--- 192.168.15.2 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3004ms
rtt min/avg/max/mdev = 0.057/0.112/0.210/0.058 ms
```

- Check **arp** table in both namespace, it should be updated.

- In namespace **red**:

```
root@vm1:~# ip netns exec red arp
Address          Hwtype  Hwaddress      Flags Mask      Iface
192.168.15.3      ether   ba:86:30:88:3b:b8 C                veth-red
```

- In namespace **blue**:

```
root@vm1:~# ip netns exec blue arp
Address          Hwtype  Hwaddress      Flags Mask      Iface
192.168.15.2      ether   de:62:4d:8e:da:b9 C                veth-blue
```

- But in Host's arp table, nothing related to namespaces will be there.

Because both are separate.

```
root@vm1:~# arp
Address          Hwtype  Hwaddress      Flags Mask      Iface
10.0.2.2          ether   52:54:00:12:35:00 C                enp0s3
10.10.10.11        ether   08:00:27:ad:99:10 C                enp0s8
```

- Try to ping the namespaces from Host.

`ping -c 4 <namespace network interface IP>`

- Ping to namespace **red**:

`ping -c 4 192.168.15.2`

```
root@vm1:~# ping -c 4 192.168.15.2
PING 192.168.15.2 (192.168.15.2) 56(84) bytes of data.
^Z
[4]+  Stopped                  ping -c 4 192.168.15.2
```

FAILED

- Ping to namespace **red**

`ping -c 4 192.168.15.3`

```
root@vm1:~# ping -c 4 192.168.15.3
PING 192.168.15.3 (192.168.15.3) 56(84) bytes of data.
^Z
[5]+  Stopped                  ping -c 4 192.168.15.3
```

FAILED

- ⌘ It is failing because there is no connection between host and namespaces.

Create a veth pair and connect that to host and namespace, then only you can ping.

[however, both end present in the host initially, just connect one to a namespace]

- ✧ To test connection between host and a namespace (let **red**), create a veth pair and connect one end to the namespace.

Another end will be by default connected to host.

- ⌘ `ip link add veth2-red type veth peer name veth2-red-host` [create veth pair]
- ⌘ `ip link set veth2-red netns red` [connect veth2-red end to namespace red]
- ⌘ `ip netns exec red ip link set veth2-red up` [make the link up in namespace]
- ⌘ `ip link set veth2-red-host up` [make the link up in host]
- ⌘ `netns exec red ip addr add 192.168.25.2/24 dev veth2-red` [assign ip in namespace]
- ⌘ `ip addr add 192.168.25.3/24 dev veth2-red-host` [assign ip in host]
- ⌘ `ping -c 4 192.168.25.2` (ping from host to namespace)

```
root@vm1:~# ping -c 4 192.168.25.2
PING 192.168.25.2 (192.168.25.2) 56(84) bytes of data.
64 bytes from 192.168.25.2: icmp_seq=1 ttl=64 time=2.54 ms
64 bytes from 192.168.25.2: icmp_seq=2 ttl=64 time=0.058 ms
64 bytes from 192.168.25.2: icmp_seq=3 ttl=64 time=0.069 ms
64 bytes from 192.168.25.2: icmp_seq=4 ttl=64 time=0.086 ms

--- 192.168.25.2 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3041ms
rtt min/avg/max/mdev = 0.058/0.688/2.542/1.070 ms
```

- ⌘ `ip netns exec red ping -c 4 192.168.25.3` (ping from namespace to host)

```
root@vm1:~# ip netns exec red ping -c 4 192.168.25.3
PING 192.168.25.3 (192.168.25.3) 56(84) bytes of data.
64 bytes from 192.168.25.3: icmp_seq=1 ttl=64 time=0.327 ms
64 bytes from 192.168.25.3: icmp_seq=2 ttl=64 time=0.062 ms
64 bytes from 192.168.25.3: icmp_seq=3 ttl=64 time=0.052 ms
64 bytes from 192.168.25.3: icmp_seq=4 ttl=64 time=0.307 ms

--- 192.168.25.3 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3005ms
rtt min/avg/max/mdev = 0.052/0.187/0.327/0.130 ms
```

- ⌘ Here we did the same thing, before connected *veth pair* from namespace to namespace; not connected *veth pair* from host to namespace.
But, we had pinged to **192.168.25.2** (not **15.2** which is previous veth's subnet)
- ⌘ You can add gateway in host
ip route add <network-ip> via <gateway-ip>
ip route add 192.168.15.0 via 192.168.25.2
but need to enable packet forwarding inside namespace **red**. Because linux doesn't forward packets by default.
- ⌘ But, one issue will be there,
we configured 192.168.15.0 via 192.168.25.2
but the namespace **blue** also have this subnet, for that also it'll send to **red** namespace because gateway is 192.168.25.2
- ⌘ We'll do this in broader picture; not namespace level;
- 々 We had 2 namespaces so one veth and directly connect both.
In case of multiple namespaces, need to create a bridge.
[inside linux, **virtual switch** can be created]
- 々 Delete all the veth pairs.
ip link delete <any one end name>
[it'll delete the other name by default]
 - ⌘ **ip netns exec red ip link delete veth-red**
 - ⌘ **ip link delete veth2-red-host**
- 々 Create a bridge
ip link add <bridge interface name> type bridge
⌘ **ip link add v-net-0 type bridge**

```
8: v-net-0: <BROADCAST,MULTICAST> mtu 1500 qdisc noop state DOWN group default qlen 1000
    link/ether fe:65:b1:8d:47:f6 brd ff:ff:ff:ff:ff:ff
```

 ⌘ [ip add]
its coming as a normal network interface only.
 - ⌘ Linux bridge is nothing but a network interface like others.
 - ⌘ The bridge belongs to the namespace where it is created.
If created in host, it belong to the host.
If created in a namespace, it belong to that namespace.

々 Make the bridge network interface UP.

`ip link set v-net-0 up`

々 Now create a **veth pair**, connect one end to the namespace **red** and another to **bridge**.

♣ `ip link add veth-red type veth peer name veth-red-br` [create veth pair]

♣ `ip link set veth-red netns red` [connect one end to namespace *red*]

♣ Now in host [`ip add`]

```
8: v-net-0: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc noqueue state DOWN group default qlen 1000
   link/ether fe:65:b1:8d:47:f6 brd ff:ff:ff:ff:ff:ff
9: veth-red-br@if10: <BROADCAST,MULTICAST> mtu 1500 qdisc noop state DOWN group default qlen 1000
   link/ether be:57:96:43:87:7c brd ff:ff:ff:ff:ff:ff link-netns red
```

♣ `ip link set veth-red-br master v-net-0` [connect another end to bridge]

[for master node's interface, use **master** <host's interface name>]

♣ `ip netns exec red ip link set veth-red up` [make namespace end up]

♣ `ip link set veth-red-br up` [make bridge end up]

々 Do the same for **blue** namespace.

♣ `ip link add veth-blue type veth peer name veth-blue-br` [create veth pair]

♣ `ip link set veth-blue netns blue` [connect one end to namespace *red*]

♣ Now in host [`ip add`]

```
11: veth-blue-br@if12: <BROADCAST,MULTICAST> mtu 1500 qdisc noop state DOWN group default qlen 1000
   link/ether 06:8d:2b:07:c6:00 brd ff:ff:ff:ff:ff:ff link-netns blue
```

♣ `ip link set veth-blue-br master v-net-0` [connect another end to bridge]

♣ `ip netns exec blue ip link set veth-blue up` [make namespace end up]

♣ `ip link set veth-blue-br up` [make bridge end up]

々 Now assign IPs to both red and blue namespace

♣ `ip netns exec red ip addr add 192.168.15.2/24 dev veth-red` [assign IP to red namespace]

♣ `ip netns exec blue ip addr add 192.168.15.3/24 dev veth-blue` [assign IP to blue namespace]

々 Now, ping from one namespace to another will be successful.

- ⌘ Ping red to blue

`ip netns exec red ping -c 4 192.168.15.3`

```
root@vm1:~# ip netns exec red ping -c 4 192.168.15.3
PING 192.168.15.3 (192.168.15.3) 56(84) bytes of data.
64 bytes from 192.168.15.3: icmp_seq=1 ttl=64 time=3.58 ms
64 bytes from 192.168.15.3: icmp_seq=2 ttl=64 time=0.103 ms
64 bytes from 192.168.15.3: icmp_seq=3 ttl=64 time=0.147 ms
64 bytes from 192.168.15.3: icmp_seq=4 ttl=64 time=0.080 ms

--- 192.168.15.3 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3044ms
rtt min/avg/max/mdev = 0.080/0.977/3.580/1.502 ms
```

- ⌘ Ping from blue to red

`ip netns exec red ping -c 4 192.168.15.3`

```
root@vm1:~# ip netns exec blue ping -c 4 192.168.15.2
PING 192.168.15.2 (192.168.15.2) 56(84) bytes of data.
64 bytes from 192.168.15.2: icmp_seq=1 ttl=64 time=0.501 ms
64 bytes from 192.168.15.2: icmp_seq=2 ttl=64 time=0.060 ms
64 bytes from 192.168.15.2: icmp_seq=3 ttl=64 time=0.107 ms
64 bytes from 192.168.15.2: icmp_seq=4 ttl=64 time=0.068 ms

--- 192.168.15.2 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3046ms
rtt min/avg/max/mdev = 0.060/0.184/0.501/0.183 ms
```

々 But, Host cannot ping either of these namespace

- ⌘ Host to namespace red

```
root@vm1:~# ping -c 4 192.168.15.2
PING 192.168.15.2 (192.168.15.2) 56(84) bytes of data.
^Z
[10]+  Stopped                  ping -c 4 192.168.15.2
```

- ⌘ Host to namespace blue

```
root@vm1:~# ping -c 4 192.168.15.3
PING 192.168.15.3 (192.168.15.3) 56(84) bytes of data.
^Z
[11]+  Stopped                  ping -c 4 192.168.15.3
```

々 Add IP address to the bridge network interface (`v-net-o`)

`ip addr add 192.168.15.5/24 dev v-net-o`

- ⌘ Because till now host doesn't know any subnet of 102.168.15.x

々 Now try to ping again.

- ⌘ Not it'll be successful

because, host has now a interface having the same subnet as that of the namespaces.

- Host to namespace red

```
root@vm1:~# ping -c 4 192.168.15.2
PING 192.168.15.2 (192.168.15.2) 56(84) bytes of data.
64 bytes from 192.168.15.2: icmp_seq=1 ttl=64 time=3.45 ms
64 bytes from 192.168.15.2: icmp_seq=2 ttl=64 time=0.373 ms
64 bytes from 192.168.15.2: icmp_seq=3 ttl=64 time=0.111 ms
64 bytes from 192.168.15.2: icmp_seq=4 ttl=64 time=0.057 ms

--- 192.168.15.2 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3059ms
rtt min/avg/max/mdev = 0.057/0.998/3.452/1.421 ms
```

- Host to namespace blue

```
root@vm1:~# ping -c 4 192.168.15.3
PING 192.168.15.3 (192.168.15.3) 56(84) bytes of data.
64 bytes from 192.168.15.3: icmp_seq=1 ttl=64 time=2.51 ms
64 bytes from 192.168.15.3: icmp_seq=2 ttl=64 time=0.248 ms
64 bytes from 192.168.15.3: icmp_seq=3 ttl=64 time=0.117 ms
64 bytes from 192.168.15.3: icmp_seq=4 ttl=64 time=0.090 ms

--- 192.168.15.3 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3004ms
rtt min/avg/max/mdev = 0.090/0.740/2.506/1.021 ms
```

- ✧ Ping from namespace to host should also work.

But to **102.168.15.5** (bridge network interface IP) only.

It'll not work for *10.10.10.10* (host (vm1) IP)

- Namespace red to host

```
root@vm1:~# ip netns exec red ping -c 4 192.168.15.5
PING 192.168.15.5 (192.168.15.5) 56(84) bytes of data.
64 bytes from 192.168.15.5: icmp_seq=1 ttl=64 time=0.894 ms
64 bytes from 192.168.15.5: icmp_seq=2 ttl=64 time=0.066 ms
64 bytes from 192.168.15.5: icmp_seq=3 ttl=64 time=0.083 ms
64 bytes from 192.168.15.5: icmp_seq=4 ttl=64 time=0.059 ms

--- 192.168.15.5 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3043ms
rtt min/avg/max/mdev = 0.059/0.275/0.894/0.357 ms
```

- Namespace blue to host.

```
root@vm1:~# ip netns exec blue ping -c 4 192.168.15.5
PING 192.168.15.5 (192.168.15.5) 56(84) bytes of data.
64 bytes from 192.168.15.5: icmp_seq=1 ttl=64 time=1.58 ms
64 bytes from 192.168.15.5: icmp_seq=2 ttl=64 time=0.080 ms
64 bytes from 192.168.15.5: icmp_seq=3 ttl=64 time=0.066 ms
64 bytes from 192.168.15.5: icmp_seq=4 ttl=64 time=0.066 ms

--- 192.168.15.5 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3038ms
rtt min/avg/max/mdev = 0.066/0.448/1.581/0.654 ms
```

- ~~To 10.10.10.10 will not work~~

```
root@vm1:~# ip netns exec red ping -c 4 10.10.10.10
ping: connect: Network is unreachable
```

```
root@vm1:~# ip netns exec blue ping -c 4 10.10.10.10
ping: connect: Network is unreachable
```

['Network is unreachable' comes when **gateway is not there**]

♪ Add gateway in the namespaces

network subnet: 10.10.10.0

gateway: 192.168.15.5 [bridge network interface's IP]

♫ **ip netns exec red ip route add 10.10.10.0/24 via 192.168.15.5** [in namespace red]

♫ **ip netns exec blue ip route add 10.10.10.0/24 via 192.168.15.5** [in namespace red]

[always add the subnet /24, otherwise it'll take 32 by default]

♪ Now try to ping host (10.10.10.10) from namespaces

♫ Namespace red to host

```
root@vm1:~# ip netns exec red ping -c 4 10.10.10.10
PING 10.10.10.10 (10.10.10.10) 56(84) bytes of data.
64 bytes from 10.10.10.10: icmp_seq=1 ttl=64 time=3.43 ms
64 bytes from 10.10.10.10: icmp_seq=2 ttl=64 time=0.074 ms
64 bytes from 10.10.10.10: icmp_seq=3 ttl=64 time=0.085 ms
64 bytes from 10.10.10.10: icmp_seq=4 ttl=64 time=0.122 ms

--- 10.10.10.10 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3037ms
rtt min/avg/max/mdev = 0.074/0.928/3.431/1.445 ms
```

♫ Namespace blue to host

```
root@vm1:~# ip netns exec blue ping -c 4 10.10.10.10
PING 10.10.10.10 (10.10.10.10) 56(84) bytes of data.
64 bytes from 10.10.10.10: icmp_seq=1 ttl=64 time=0.917 ms
64 bytes from 10.10.10.10: icmp_seq=2 ttl=64 time=0.097 ms
64 bytes from 10.10.10.10: icmp_seq=3 ttl=64 time=0.063 ms
64 bytes from 10.10.10.10: icmp_seq=4 ttl=64 time=0.075 ms

--- 10.10.10.10 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3036ms
rtt min/avg/max/mdev = 0.063/0.288/0.917/0.363 ms
```

♪ Now try to ping **vm2** (10.10.10.11) from the namespaces.

It'll ping (because gateway is there) but response will not come.

♫ Namespace red to vm2

```
root@vm1:~# ip netns exec red ping -c 4 10.10.10.11
PING 10.10.10.11 (10.10.10.11) 56(84) bytes of data.
^Z
[14]+  Stopped                  ip netns exec red ping -c 4 10.10.10.11
```

♫ Namespace blue to vm2

```
root@vm1:~# ip netns exec blue ping -c 4 10.10.10.11
PING 10.10.10.11 (10.10.10.11) 56(84) bytes of data.
^Z
[15]+  Stopped                  ip netns exec blue ping -c 4 10.10.10.11
```


- ⌘ It is because, packet is going to vm2 (10.10.10.11), but it doesn't know about **192.168.15.x** because it belongs to private network of **vm1** (host).
- ⌘ Either add a gw in **vm2** or enable **NAT** in vm1.

🔗 Option-1: adding gateway in **vm2**

network: 192.168.15.0

gateway: 10.10.10.10 (vm1 / host)

- ⌘ **ip route add 192.168.15.0/24 via 10.10.10.10 [vm2]**

```
root@vm2:~# ip route
default via 10.0.2.2 dev enp0s3 proto dhcp src 10.0.2.15 metric 100
10.0.2.0/24 dev enp0s3 proto kernel scope link src 10.0.2.15 metric 100
10.0.2.2 dev enp0s3 proto dhcp scope link src 10.0.2.15 metric 100
10.10.10.0/24 dev enp0s8 proto kernel scope link src 10.10.10.11
192.168.0.1 via 10.0.2.2 dev enp0s3 proto dhcp src 10.0.2.15 metric 100
192.168.15.0/24 via 10.10.10.10 dev enp0s8
```

[vm2]

[ip route]

- ⌘ Even now ping from namespace to vm2 or vice-versa will not work. Because, **IP forwarding is disable in vm1** (host)

cat /proc/sys/net/ipv4/ip_forward

0: disabled

1: enabled

- ⌘ **vm1 OS is acting as the gateway and router for different networks.**

10.10.10.x network

& 192.168.15.x network.

- ⌘ A router's main job is receive packet from a network, and forward that to another network.

Here, **vm1 OS** is acting as a router, receiving from one network and forwarding to another. So, packet forwarding needs to be enabled.

- ⌘ Enable ip forwarding

echo 1 > /proc/sys/net/ipv4/ip_forward

- ⌘ Now ping will work

```
root@vm2:~# ping 192.168.15.2
PING 192.168.15.2 (192.168.15.2) 56(84) bytes of data.
64 bytes from 192.168.15.2: icmp_seq=1 ttl=63 time=4.68 ms
64 bytes from 192.168.15.2: icmp_seq=2 ttl=63 time=1.55 ms
64 bytes from 192.168.15.2: icmp_seq=3 ttl=63 time=1.31 ms
^Z
[8]+  Stopped                  ping 192.168.15.2
```

[vm2 to

namespace]


```

root@vm1:~# ip netns exec red ping 10.10.10.11
PING 10.10.10.11 (10.10.10.11) 56(84) bytes of data.
64 bytes from 10.10.10.11: icmp_seq=1 ttl=63 time=1.90 ms
64 bytes from 10.10.10.11: icmp_seq=2 ttl=63 time=0.902 ms
64 bytes from 10.10.10.11: icmp_seq=3 ttl=63 time=2.23 ms
64 bytes from 10.10.10.11: icmp_seq=4 ttl=63 time=1.20 ms
^Z
[20]+  Stopped                  ip netns exec red ping 10.10.10.11

```

[namespace t

vm2]

- ♣ In this case, **vm2** can directly communicate to **namespace** (of **vm1**).

♪ Option-2: using NAT

- ♣ Previous option is not preferable because if multiple systems will be there, in each system we need to set route gateway.
- ♣ If we enable NAT in **vm1**, it'll update the source IP before the packet exits.

- ♣ Packet before NAT:

src IP: 192.168.15.2 (red namespace)

dest IP: 10.10.10.11 (vm2)

- ♣ Packet after NAT:

src IP: 10.10.10.10 (host/vm1 IP)

dest IP: 10.10.10.11 (vm2) --> it doesn't change.

- ♣ Now the packet can go to **vm2** easily, and vm2 as well send back the response.

src IP: 10.10.10.11

dest IP: 10.10.10.10 (because this was the source IP received at vm2)

- ♣ vm1 (host) will receive the packet. And from the NAT table it knows that this packet belong to 192.168.15.2 (red namespace).

Now it updates the packet's dest IP

src IP: 10.10.10.11 (vm2, doesn't change)

dest IP: 192.158.15.2 (red namespace)

- ♣ First delete the created configs (vm2 route gateway).

Don't disable IP packet forwarding (/proc/sys/net/ipv4/ip_forward) because NAT requires this.

- ♣ **iptables -t nat -A POSTROUTING -o <network interface name> -j**

MASQUERADE

-t: use NAT table

POSTROUTING: modify packet before leaving

<network interface name>: network interface connected to vm2

```
3: enp0s8: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu
    link/ether 08:00:27:c3:54:e8 brd ff:ff:ff:ff:
    inet 10.10.10.10/24 brd 10.10.10.255 scope g
        valid_lft forever preferred_lft forever
    inet6 fe80::a00:27ff:fec3:54e8/64 scope link
        valid_lft forever preferred_lft forever
```

[in my case **enpos8**]

MASQUERADE: replace source IP with host/router IP

[for incoming traffic, destination ip will be updated (host IP to namespace's IP). it looks like DNAT but it is not. Kernel automatically reverse NAT mapping because it remembers previously source IP was changed]

⚡ **iptables -t nat -A POSTROUTING -o enpos8 -j MASQUERADE**

⚡ Now, from namespace to vm2 can be easily accessible.

```
root@vm1:~# ip netns exec red ping -c 4 10.10.10.11
PING 10.10.10.11 (10.10.10.11) 56(84) bytes of data.
64 bytes from 10.10.10.11: icmp_seq=1 ttl=63 time=3.74 ms
64 bytes from 10.10.10.11: icmp_seq=2 ttl=63 time=1.83 ms
64 bytes from 10.10.10.11: icmp_seq=3 ttl=63 time=1.90 ms
64 bytes from 10.10.10.11: icmp_seq=4 ttl=63 time=1.03 ms

--- 10.10.10.11 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3009ms
rtt min/avg/max/mdev = 1.027/2.123/3.736/0.992 ms
```

[red to vm2]

```
root@vm1:~# ip netns exec blue ping -c 4 10.10.10.11
PING 10.10.10.11 (10.10.10.11) 56(84) bytes of data.
64 bytes from 10.10.10.11: icmp_seq=1 ttl=63 time=3.14 ms
64 bytes from 10.10.10.11: icmp_seq=2 ttl=63 time=1.29 ms
64 bytes from 10.10.10.11: icmp_seq=3 ttl=63 time=1.49 ms
64 bytes from 10.10.10.11: icmp_seq=4 ttl=63 time=0.906 ms

--- 10.10.10.11 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3005ms
rtt min/avg/max/mdev = 0.906/1.707/3.140/0.853 ms
```

[blue to vm2]

⚡ But, **vm2** to **namespace** will not be accessible.

Namespace to vm2:

when vm2 receives the packet, *src ip: vm1's ip*, *dest ip: vm2's ip*
so, vm2 also sends the packet, *src ip: vm2's ip*, *dest ip: vm1's ip*
at the end, vm1 forward the packet to namespace.

[in this scenario, vm2 is sending packet to vm1 only, not to namespace]

⚡ But, directly if you try to access namespace from vm2 it'll not work because vm2 doesn't know about namespace.

If you want this only, then **route** needs to be added in **vm2** .

- ⌘ **Port forwarding** can be used in this case.

Vm2 will send packet to vm1 at a particular port. Vm1 will forward the packet to namespace.

It is **DNAT**.

- ⌘ **iptables -t nat -A PREROUTING -p tcp --dport 8080 -j DNAT --to-destination 192.168.15.2:80**

Packets arriving on port 8080 should be redirected to namespace 192.168.15.2:80

- ⌘ I ran a simple nodejs in namespace *red* on port 80,
curl <http://10.10.10.10:8080> [from vm2]

```
root@vm2:~# curl http://10.10.10.10:8080
Hello from Node.js server on port 80
```